Министерство науки и вышего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

« Московский педагогический государственный университет »

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №7

Выполнил студент группы ЗОМ34-ПВШ2616: **Золотухин Андрей Александрович**

Ссылка на репозиторий: https://github.com/CorgiPuppy/dslp-labs

Принял: Жданов Сергей Александрович

Дата сдачи: ..26

Москва

2026

СОДЕРЖАНИЕ

1. Задание 1 3
   1.1. Ознакомление с примерами из лекции 3
2. Задание 2 11

1. Задание 1

1.1. Ознакомление с примерами из лекции

```
// pairArray.cpp
// C++ программа для создания пар всех чётных чисел, присутствующих в заданном массиве
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович

/* Для заданного  целочисленного массива вывести все пары,
 * которые содержат четные числа. Если пар четных чисел нет,
 * то вывести -1.
 */

#include <iostream>
#include <vector>

// замена, чтобы не писать очень длинный тип данных
typedef std::pair<int, int> pairs;

// функция для вывода массива пар
void display_pairs(const std::vector<pairs>& s){
  bool found = false;

   // вывести, если есть элементы в массиве
  for (int i = 0; i < s.size(); i++) {
    found = true;
    std::cout << "(" << s[i].first << ", "
         << s[i].second << ")"
         << " ";
  }

  // в массиве нет элементов
  // (по идее не вызовется никогда, так как мы это проверяем ещё в main. То есть мы вызываем эту функцию только когда чётные числа существуют, чтобы отобразить эти пары)
  if (not found)
      std::cout << "There are no pairs of even numbers in the array.\n ";
}

int main() {
  int data;
  std::vector<int> v;
  std::vector<pairs> s;

  std::cout << "Enter numbers via spaces: "; // добавил информацию о том, что
```

```
ждёт программа от пользователя
  // ввод строки данных из потока до конца строки
  while (std::cin.peek() != '\n') {
    std::cin >> data;
    v.push_back(data);
  }

  // анализ данных
  bool even_numbers = false;
  for (int i = 0; i < v.size(); i++) { // исправил проход до v.size()-1 вместо
v.size()-2
    for (int j = i; j < v.size(); j++) { // исправил инициализацию j как i
вместо i+1, чтобы затрагивать одинаковые чётные числа (типа 2,2 или 4,4, не
упуская их, так как это тоже считается парой)
      // проверка чисел на четность
      if (v[i] % 2 == 0 && v[j] % 2 == 0) {
        // создать пару из четных чисел
        pairs x = std::make_pair(v[i], v[j]);
        even_numbers = true;

        // и сохранить в вектор
        s.push_back(x);
      }
    }
  }

  // отобразить результат
  if (even_numbers) {
    std::cout << "Pairs of even numbers: "; // для понятного вывода
    display_pairs(s);
    std::cout << std::endl;
  }
  else std::cout << -1 << std::endl;

  // очистить вектор
  s.clear();
}
```

```
Enter numbers via spaces: 1 2 3 4 5 6 7 8 9 10
Pairs of even numbers: (2, 2) (2, 4) (2, 6) (2, 8) (2, 10) (4, 4) (4, 6) (4,
8) (4, 10) (6, 6) (6, 8) (6, 10) (8, 8) (8, 10) (10, 10)
```

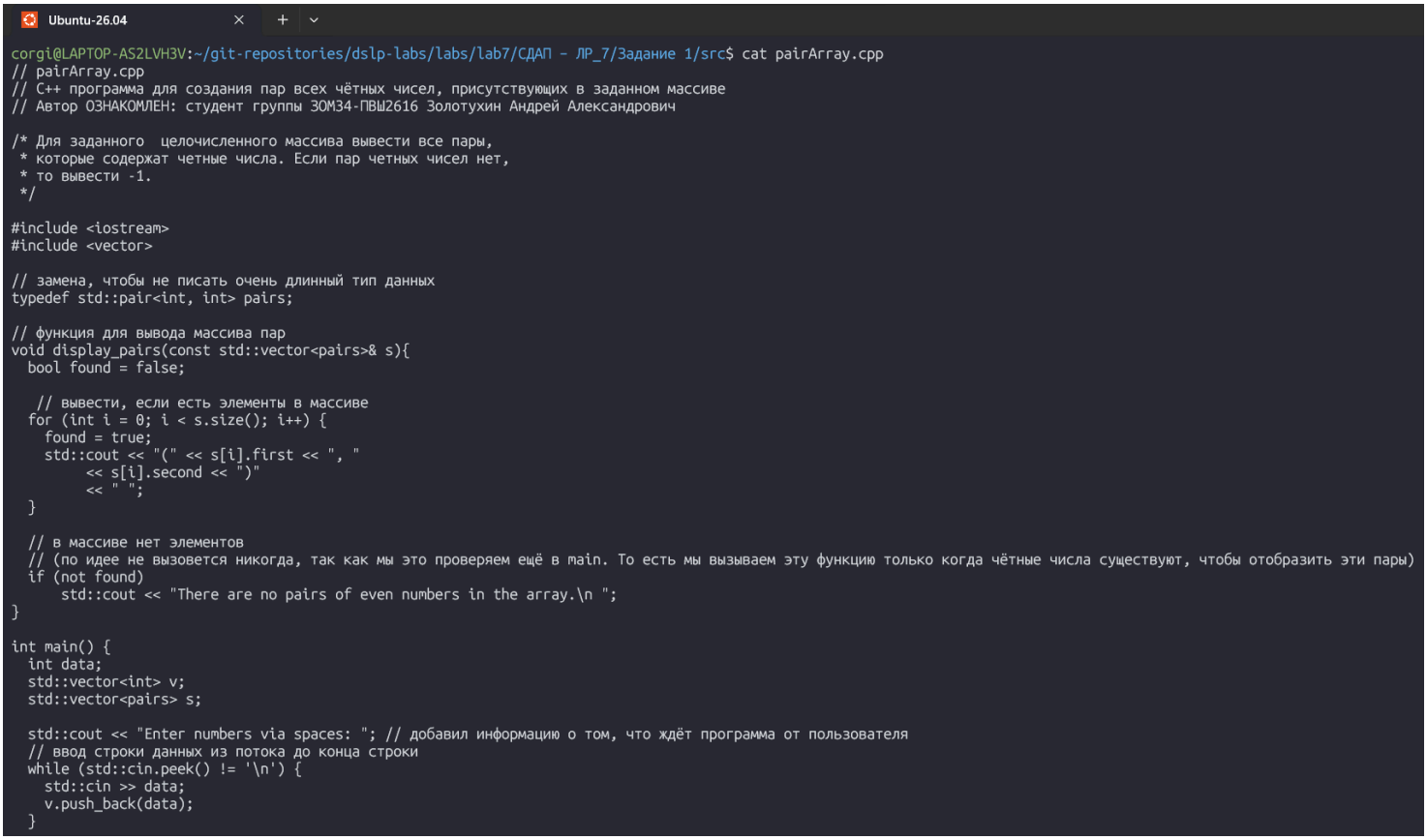


Рис. 1: Авторская иллюстрация программы pairArray.cpp

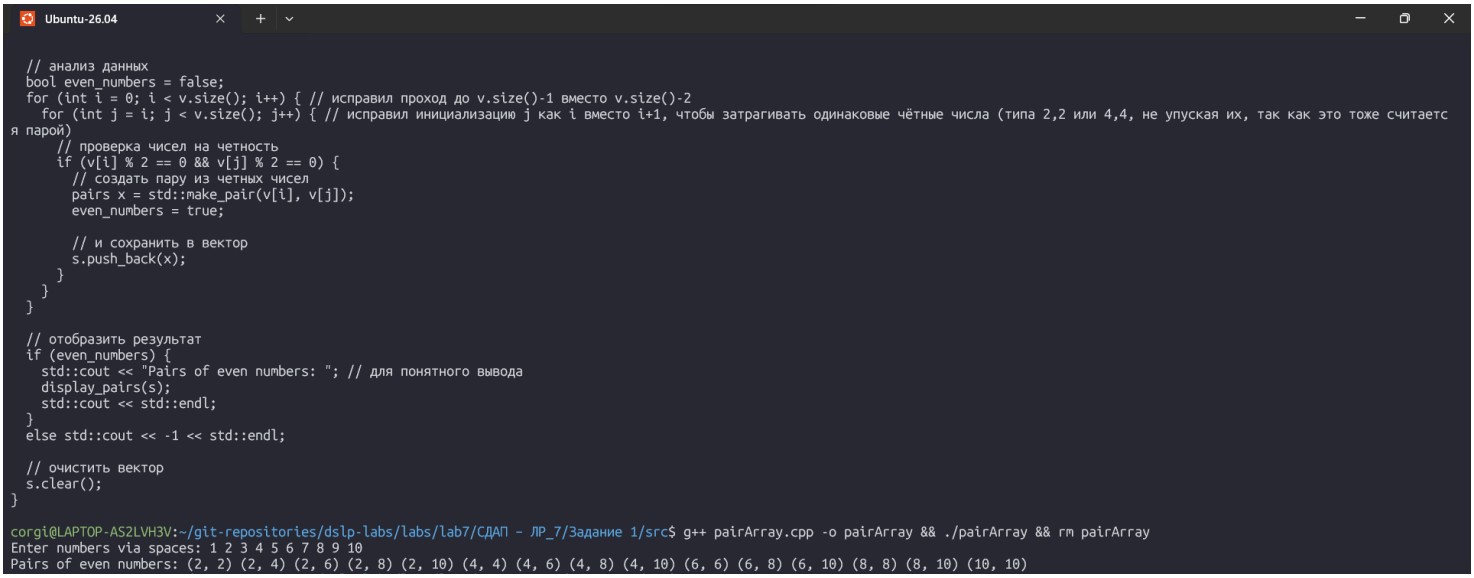


Рис. 2: Авторская иллюстрация программы pairArray.cpp (продолжение)

```cpp
// pairCompare.cpp
// C++ программа для сравнения пар чисел
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

#include <iostream>
#include <utility>
```

```
// для простоты объявления
typedef std::pair<int, int> pairs;

int main() {
  pairs pair1 = std::make_pair(1, 12);
  pairs pair2 = std::make_pair(9, 12);

  // вывод пар
  std::cout << "Pair pair1 = (" << pair1.first << ", " << pair1.second << ")"
<< std::endl;
  std::cout << "Pair pair2 = (" << pair2.first << ", " << pair2.second << ")"
<< std::endl << std::endl;

  // все возможные сравнения двух пар
  std::cout << "Comparison (pair1 == pair2)" << " returns " << (pair1 ==
pair2) << std::endl;
  std::cout << "Comparison (pair1 != pair2)" << " returns " << (pair1 !=
pair2) << std::endl;
  std::cout << "Comparison (pair1 >= pair2)" << " returns " << (pair1 >=
pair2) << std::endl;
  std::cout << "Comparison (pair1 <= pair2)" << " returns " << (pair1 <=
pair2) << std::endl;
  std::cout << "Comparison (pair1 > pair2)" << " returns " << (pair1 > pair2)
<< std::endl;
  std::cout << "Comparison (pair1 < pair2)" << " returns " << (pair1 < pair2)
<< std::endl;

  return 0;
}
```

```
Pair pair1 = (1, 12)
Pair pair2 = (9, 12)

Comparison (pair1 == pair2) returns 0
Comparison (pair1 != pair2) returns 1
Comparison (pair1 >= pair2) returns 0
Comparison (pair1 <= pair2) returns 1
Comparison (pair1 > pair2) returns 0
Comparison (pair1 < pair2) returns 1
```

```
// pairSTL.cpp
// C++ программа для иллюстрации контейнера pair STL
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович
```

```cpp
#include <iostream>
#include <utility>
#include <string>

int main() {
  // определение и инициализация пар
  std::pair<std::string, int> g1;
  std::pair<std::string, int> g2("Quiz", 3);
  std::pair<std::string, int> g3(g2);
  std::pair<int, int> g4(5, 10);

  g1 = std::make_pair(std::string("Site"), 1);
  g2.first = ".com";
  g2.second = 2;

  std::cout << "Pair g1 = (" << g1.first << ", " << g1.second << ")" <<
std::endl << std::endl;
  std::cout << "Pair g2 = (" << g2.first << ", " << g2.second << ")" <<
std::endl << std::endl;

  std::cout << "Pair g3 = (" << g3.first << ", " << g3.second << ").\n" <<
"\tThis pair was created as a copy of the g2 pair." << std::endl << std::endl;

  std::cout << "Pair g2 = (" << g2.first << ", " << g2.second << ").\n" <<
"\tThe value of this pair has been changed since initialization." << std::endl
<< std::endl;

  std::cout << "Pair g4 = (" << g4.first << ", " << g4.second << ").\n" <<
"\tSum of key and value pair g4 =" << g4.first + g4.second << "." << std::endl
<< std::endl;

  std::cout << "Let\'s find the concatenation of the symbolic keys of the
pairs g1, g2 и g3:\n" << g1.first + g3.first + g2.first << std::endl <<
std::endl;

  std::cout << "You can change pairs, but their type must be the same:" <<
std::endl;
  std::cout << "\tBefore the exchange of pairs:\n" << "\t\tg1 = ("<< g1.first
<< ", "<<g1.second<<")\n " << "\t\tg2 = (" << g2.first << ", " << g2.second <<
")" << std::endl;

  // обмен элементами
  std::swap(g1, g2);
  std::cout << "\tAfter the exchange:\n" << "\t\tg1 = ("<< g1.first << ", " <<
g1.second << ")\n" << "\t\tg2 = (" << g2.first <<", " << g2.second << ")"<<
```

```
std::endl;

  return 0;
}
```

```
Pair g1 = (Site, 1)

Pair g2 = (.com, 2)

Pair g3 = (Quiz, 3).
  This pair was created as a copy of the g2 pair.

Pair g2 = (.com, 2).
  The value of this pair has been changed since initialization.

Pair g4 = (5, 10).
  Sum of key and value pair g4 =15.

Let's find the concatenation of the symbolic keys of the pairs g1, g2 и g3:
SiteQuiz.com

You can change pairs, but their type must be the same:
  Before the exchange of pairs:
    g1 = (Site, 1)
    g2 = (.com, 2)
  After the exchange:
    g1 = (.com, 2)
    g2 = (Site, 1)
```

```
// testSet1.cpp
// C++ программа-пример использования контейнера set
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

#include <iostream>
#include <vector>
#include <set>
#include <iterator>

int main() {
  int len_v;
  std::set<int> s; // зададим  s как контейнер set
  int v[] = {7, 6, 1, 4, 2, 9, 10, 5, 3, 7, 5}; // массив данных
```

```
  std::cout << "Input data: ";
  len_v = (sizeof(v) / sizeof(*v)); // вычисляем длину массива
  for (int i = 0; i < len_v; i++)  // вывод элементов массива данных
    std::cout << v[i] << " ";
  std::cout << std::endl;

  s.insert(v, v + len_v); // вставляем данные в контейнер set

  std::cout << "Set elements: ";
  // вывод элементов множества в стандартный поток вывода
  std::copy(s.begin(),s.end(), std::ostream_iterator<int>(std::cout," "));
  std::cout << "\n";

  return 0;
}
```

```
Input data: 7 6 1 4 2 9 10 5 3 7 5
Set elements: 1 2 3 4 5 6 7 9 10
```

```
// testSet1.cpp
// C++ программа-пример операций на множествах
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

#include <iostream>
#include <iterator>
#include <set>
#include <algorithm>

// для удобства объявления
typedef std::set<int> settype;

// переопределение операций
settype operator* (const settype &S, const settype &T ) {
  settype I; // пересечение множеств
  set_intersection(S.begin(),S.end(), T.begin(),T.end(),
inserter(I,I.begin()));
  return I;
}

settype operator+ (const settype &S, const settype &T ) {
  settype U;  // объединение множеств
  set_union(S.begin(),S.end(),
        T.begin(),T.end(),
```

```
        inserter(U,U.begin()));
  return U;
}

// функция вывода множества
void out_set(const char *set_name, settype S) {
  std::cout << " " << set_name;
  std::copy(S.begin(), S.end(), std::ostream_iterator<int>(std::cout, " "));
  std::cout << std::endl;
}

int main() {
  int aS[3] = {10, 20, 30}, aT[4] = {20, 30, 40, 50}; // массивы исходных
данных
  settype S(aS, aS + 3), T(aT, aT + 4); // создать множества

  std::cout << "Input sets:\n";
  out_set("\tS = ", S);
  out_set("\tT = ", T);

  std::cout << "Intesection sets S and T:\n";
  out_set("\tS * T = ", S * T);

  std::cout <<"Union sets S and T:\n";
  out_set("\tS + T = ", S + T);

  return 0;
}
```

```
Input sets:
  S = 10 20 30
  T = 20 30 40 50
Intesection sets S and T:
  S * T = 20 30
Union sets S and T:
  S + T = 10 20 30 40 50
```

2. Задание 2